

Evaluating Classifiers

Errors, metrics, and thresholds

Which mistakes matter for this application?

Foundations of Machine Learning · Model Evaluation

A fraud detector with 99% accuracy

A payment service sees 9,900 legitimate transactions and 100 fraudulent transactions.

Its new detector lets every transaction through.

99% accuracy

Would you deploy it?

The action behind a positive prediction

For our payment example:

Positive class

Fraud.

Positive prediction

Flag the transaction for review.

Negative prediction

Allow it without review.

Is flagging for review the same decision as blocking a payment?

One cohort for the worked example

1,000 transactions

100 fraudulent
900 legitimate

At the current threshold

170 transactions get flagged.
80 of those really are fraud.

What happened to the rest?

20 fraud cases slipped through.
810 legitimate payments passed.

Illustrative validation data, not a real fraud dataset.

Four possible outcomes

Outcome	Payment example	Count
True positive, TP	Fraud correctly flagged	80
False positive, FP	Legitimate payment flagged	90
False negative, FN	Fraud allowed through	20
True negative, TN	Legitimate payment allowed through	810

Which errors cost the payment service money? Which affect its customers?

The confusion matrix

Actual / predicted

Legitimate

Fraud

Legitimate

810

TN

90

FP

Fraud

20

FN

80

TP

Rows = actual class. Columns = predicted class.

Row totals: 900 legitimate and 100 fraud.

Column totals: 830 allowed and 170 flagged.

Classification terminology

Positive / negative

In these terms, this names the predicted class.

True / false

This tells us whether the prediction agrees with the ground truth.

Term	Prediction	Ground truth
True Positive (TP)	Positive	Positive
False Positive (FP)	Positive	Negative
True Negative (TN)	Negative	Negative
False Negative (FN)	Negative	Positive

Accuracy: how often were we correct?

Actual / predicted	Legitimate	Fraud
Legitimate	810 TN	90 FP
Fraud	20 FN	80 TP

$$\text{Accuracy} = \frac{TP + TN}{N}$$

$$\frac{80 + 810}{1000} = 0.89$$

89% accuracy

Without any additional information,
do we know if this is the best
possible classifier?

Precision: how trustworthy are the alerts?

Actual / predicted	Legitimate	Fraud
Legitimate	810 TN	90 FP
Fraud	20 FN	80 TP

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\frac{80}{80 + 90} = 0.471$$

47.1% of flagged payments really are fraud.

The denominator is the 170 alerts.

You can trust the fraud alerts **47.1% of the time.**

Recall: how much fraud did we catch?

Actual / predicted	Legitimate	Fraud
Legitimate	810 TN	90 FP
Fraud	20 FN	80 TP

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\frac{80}{80 + 20} = 0.80$$

We caught 80% of the fraud.

The denominator is the 100 real fraud cases.

20% of fraud goes undetected.

Precision and recall in everyday language

Question	Group we look at	Metric
How many of our alerts were real?	Predicted positives	Precision
How many real cases did we find?	Actual positives	Recall

Same accuracy, different mistakes

Model A

Actual / predicted	Legitimate	Fraud
Legitimate	840 TN	60 FP
Fraud	20 FN	80 TP

Precision 57.1% · Recall 80%

Model B

Actual / predicted	Legitimate	Fraud
Legitimate	870 TN	30 FP
Fraud	50 FN	50 TP

Precision 62.5% · Recall 50%

Both have 92% accuracy on the same class totals.

False alerts or missed fraud?

Model A

Actual / predicted	Legitimate	Fraud
Legitimate	890 TN	10 FP
Fraud	40 FN	60 TP

Precision **85.7%** · Recall **60%**

Model B

Actual / predicted	Legitimate	Fraud
Legitimate	700 TN	200 FP
Fraud	0 FN	100 TP

Precision **33.3%** · Recall **100%**

Would you prefer more false fraud alerts with no missed fraud, or fewer false alerts with more fraud going undetected?

Illustrative predictions on the same 1,000 transactions: 100 fraud, 900 legitimate.

F1: a combined precision–recall summary

$$F_1 = \frac{2PR}{P + R}$$

$$F_1 = \frac{2TP}{2TP + FP + FN}$$

Our original model: **F1 = 0.593**

Precision = 1.00, recall = 0.01

Arithmetic mean = 0.505

F1 ≈ 0.0198

A very weak precision or recall pulls F1 down.

A high F1 does not settle the decision

F1 combines precision and recall with equal emphasis.

It leaves out:

Missed fraud

The cost of each fraud case that slips through.

Customer disruption

The cost of flagging a legitimate payment.

Review capacity

How many alerts the team can handle.

What objective would you choose if reviewers can handle only 100 alerts per day?

Scores become decisions through a threshold

The model gives each transaction a score.

$$\hat{y} = \begin{cases} 1 & s \geq t \\ 0 & s < t \end{cases}$$

A score of **0.42** is flagged at **t = 0.30**, but allowed at **t = 0.50**.

The score stays the same. The decision changes.

A score need not be a calibrated probability.

A small set of transactions

Transaction	Score	Actual class	Flagged at $t = 0.50$?
A	0.90	Fraud	Yes
B	0.80	Legitimate	Yes
C	0.60	Fraud	Yes
D	0.40	Fraud	No
E	0.20	Legitimate	No
F	0.10	Legitimate	No

What changes if the threshold moves to 0.40?

What lowering the threshold guarantees

On a fixed set of scores, lowering the threshold:

- Adds positive predictions, or leaves them unchanged.
- Increases or preserves both TP and FP.
- Increases or preserves recall and false-positive rate.

Precision can rise or fall

It depends on the newly flagged cases.

In the six-transaction example, adding one fraud case improved both precision and recall.

Threshold lab: the same fraud model

Flag when score $\geq$ 0.50



t = 0.80

t = 0.50

t = 0.20

Actual /
predicted

Legitimate

Fraud

Legitimate

810

TN

90

FP

Fraud

20

FN

80

TP

Alerts: 170 of 1,000

Precision: 47.1%

Recall: 80.0%

F1: 0.593

Accuracy: 89.0%

Illustrative validation cohort. Scores occur at 0.90, 0.70, 0.50, 0.30, and 0.10.

Three operating points

Threshold	TP	FP	FN	Alerts	Precision	Recall
0.80	50	10	50	60	83.3%	50%
0.50	80	90	20	170	47.1%	80%
0.20	95	200	5	295	32.2%	95%

Same model. Same cohort. Different decisions.

Which threshold would you choose? What information is missing?

Threshold choice depends on the action

Application and action	Consequence of a false positive	Consequence of a false negative
Fraud: request verification	Customer friction	Fraud passes without review
Spam: quarantine the email	A real message is hidden	Spam reaches the inbox
Safety: send an item for inspection	Extra inspection	A hazardous item passes

The positive class and the response must be explicit.

The cost of missed fraud changes the choice

Suppose a false alert costs \$5 and a missed fraud case costs \$100.

$$\text{Total error cost}(t) = 5 FP(t) + 100 FN(t)$$

Threshold	False-alert cost	Missed-fraud cost	Total
0.80	\$50	\$5,000	\$5,050
0.50	\$450	\$2,000	\$2,450
0.20	\$1,000	\$500	\$1,500

Illustrative constant costs. Lowest cost among these three candidates: $t = 0.20$.

Changing the action changes the preferred threshold

Now a false decline costs \$100, while a missed fraud case costs \$20.

$$\text{Total error cost}(t) = 100 FP(t) + 20 FN(t)$$

Threshold	False-decline cost	Missed-fraud cost	Total
0.80	\$1,000	\$1,000	\$2,000
0.50	\$9,000	\$400	\$9,400
0.20	\$20,000	\$100	\$20,100

Same scores and counts. Lowest cost among these candidates: $t = 0.80$.

A capacity limit can rule out a threshold

The review team can handle **at most 100 alerts** per cohort.

Threshold	Alerts	Fraud caught	Within capacity?
0.80	60	50	Yes
0.50	170	80	No
0.20	295	95	No

Among these candidates, $t = 0.80$ is the only feasible choice.

Other policies include reviewing the highest-scoring cases or expanding capacity.

A threshold objective needs a complete sentence

Examples of useful objectives:

Cost objective

Minimize error cost using agreed costs for false alerts and missed fraud.

Capacity constraint

Maximize recall while generating no more than 100 review alerts.

Detection requirement

Maximize precision while maintaining at least 90% recall.

For automatic spam quarantine, what objective would you propose and why?

Threshold tuning belongs on validation data

Stage	What we do
Training	Fit the model and preprocessing
Validation	Select the model, threshold, and decision rule
Test	Evaluate the locked model and decision rule

The test set stays out of threshold selection.

With limited data, use an appropriate cross-validation procedure.

ROC needs one more rate

Actual / predicted	Legitimate	Fraud
Legitimate	810 TN	90 FP
Fraud	20 FN	80 TP

$$\text{FPR} = \frac{FP}{FP + TN}$$

$$\frac{90}{90 + 810} = 0.10$$

10% of legitimate payments are flagged.

Recall = TPR = 80%

Each threshold gives a point on the ROC curve

Threshold	FPR	Recall / TPR
0.80	0.011	0.50
0.50	0.100	0.80
0.20	0.222	0.95

Sweeping every distinct score produces the full curve.



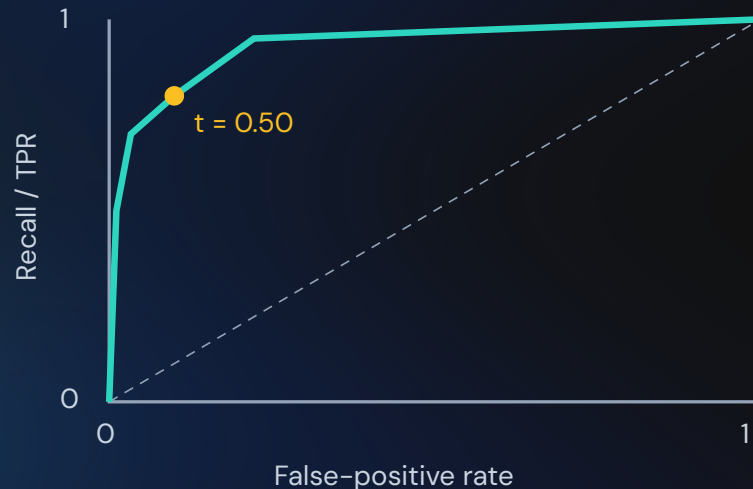
Curve calculated from all five score bins in the running example.

ROC-AUC summarizes the ranking

ROC-AUC = 0.931 for our score ranking.

AUC gives credit when a randomly chosen fraud case scores above a randomly chosen legitimate case.

Ties receive half credit.



AUC does not choose the operating threshold.

Rare positives can create many false alerts

A different deployment population has 10 fraud cases and 9,990 legitimate payments.

The detector catches 8 fraud cases and raises 92 false alerts.

Measure	Calculation	Value
Recall	$8/10$	80%
False-positive rate	$92/9,990$	0.92%
Precision	$8/(8+92)$	8%

A small false-positive rate can still produce a large share of false alerts.

Precision-recall curves describe alert quality

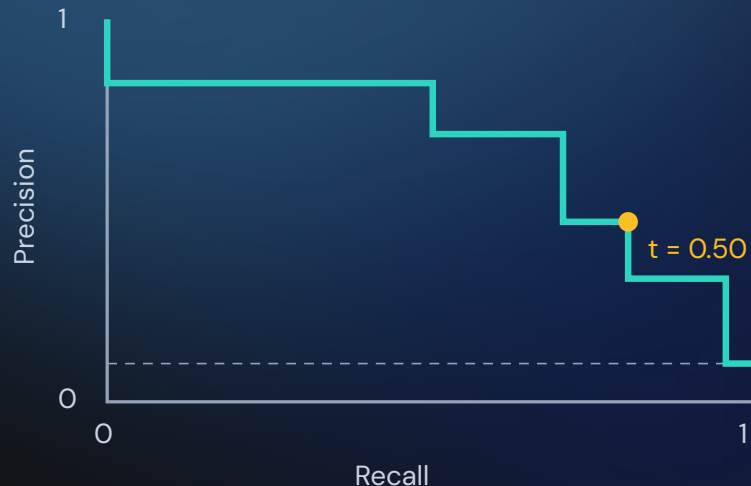
Back to our 1,000 transactions:

- Horizontal axis: recall.
- Vertical axis: precision.
- Each operating point uses a threshold.

Average precision

0.657 for this score ranking.

Dashed baseline: 10% prevalence in this cohort.



Which evidence answers which question?

Question	Evidence
What mistakes occur at this threshold?	Confusion matrix
How reliable are positive predictions?	Precision
How many real cases do we catch?	Recall
How do precision and recall combine?	F1
How well do scores rank the classes?	ROC-AUC and precision-recall curve / AP
Should we take this action?	Costs, constraints, and validation results

A threshold in Python

```
import numpy as np
from sklearn.metrics import confusion_matrix, precision_score, recall_score

# The fitted model's classes are [0, 1]; 1 means fraud.
fraud_column = np.flatnonzero(model.classes_ == 1).item()
s_val = model.predict_proba(X_val)[: , fraud_column]
t = 0.20 # chosen using validation costs or constraints
pred_val = (s_val ≥ t).astype(int)

print(confusion_matrix(y_val, pred_val, labels=[0, 1]))
print(precision_score(y_val, pred_val, zero_division=0))
print(recall_score(y_val, pred_val, zero_division=0))
```

After selection, apply the **locked threshold** to test scores.

Your deployment recommendation

Your review team can handle **200 alerts** per 1,000 transactions.

Stakeholders require **at least 80% recall**.

Choose among **$t = 0.80, 0.50$, and 0.20** .

Which meets both requirements? What would you report?

$t = 0.50$: 170 alerts, 80% recall, 47.1% precision.

Report the confusion matrix, threshold, cohort, and constraints.

Optional extensions

More ways to examine classifier behavior

- Specificity and negative predictive value

- Multiclass matrices and averaging

- Probability calibration

- Prevalence changes

- Thresholds from calibrated probabilities

The core lesson ends with the deployment recommendation.

Specificity and negative predictive value

Measure	Question	Running example
Specificity	Of actual negatives, how many pass correctly?	810/900 = 90%
Negative predictive value	Of negative predictions, how many are correct?	810/830 = 97.6%

$$\text{Specificity} = \frac{TN}{TN + FP} = 1 - \text{FPR}$$

$$\text{NPV} = \frac{TN}{TN + FN}$$

A multiclass confusion matrix

A fictional wildlife camera classifies **deer, fox, and dog**.

Actual / predicted	Deer	Fox	Dog
Deer	45	3	2
Fox	1	12	7
Dog	0	4	26

Fox precision = $12/19 = 63.2\%$

Fox recall = $12/20 = 60\%$

Which confusion would you investigate first?

Macro, weighted, and micro summaries

Recall summary	How it combines classes	Wildlife example
Macro	Mean of the three class recalls	78.9%
Weighted	Weight each recall by actual class size	83%
Micro	Pool class TP and FN counts first	83%

Deer recall: 90% · Fox recall: 60% · Dog recall: 86.7%

Always inspect important classes directly.

Ranking and probability calibration

Model A says 0.80 for a group of transactions.

Model B says 0.99 for exactly the same group.

If only 80% of those transactions are fraud, Model A better matches the observed frequency.

The two models can still have identical rankings and ROC-AUC.

Reliability diagrams compare predicted probabilities with observed frequencies.

A prevalence change alters precision

Keep recall at 80% and FPR at 10%.

Population	TP	FP	Precision
100 fraud, 900 legitimate	80	90	47.1%
10 fraud, 990 legitimate	8	99	7.5%

The conditional rates stayed fixed, but alert quality changed.

Recheck thresholds and workload in the deployment population.

A cost threshold from calibrated probabilities

Assume p is a calibrated fraud probability and each error type has a constant cost.

Allow: expected cost = $p C_{FN}$

Flag: expected cost = $(1 - p) C_{FP}$

Flag when:

$$p \geq \frac{C_{FP}}{C_{FP} + C_{FN}}$$

This simplified rule assumes zero cost for correct decisions and no capacity constraint.

References and example data

Scikit-learn: metrics and scoring

Scikit-learn: decision threshold tuning

Scikit-learn: precision-recall example

Scikit-learn: probability calibration

All numerical scenarios in this presentation are illustrative. The supplied Python example reproduces the fraud cohort, threshold results, ROC-AUC, and average precision.